

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 15, Sesión 1

Introducción y Preparación para la Prueba

Datos Proporcionados para los Ejercicios

Ejercicio 1: Movimiento Rectilíneo Uniforme (`np.polyfit`)

Solución Ejercicio 1

Ejercicio 2: Ley de Gravitación Universal (`curve_fit`)

Solución Ejercicio 2

Ejercicio 3: Análisis del Algoritmo Selection Sort

Solución Ejercicio 3

Consolidación y Consejos para la Prueba

Conclusiones

Introducción y Preparación para la Prueba

Introducción y Preparación para la Prueba $\ni$ Objetivos de la Sesión de Preparación

- **Practicar** con ejercicios similares a los de la prueba
- **Consolidar** el uso de `np.polyfit` y `curve_fit`
- **Aplicar** análisis de algoritmos con instrumentación
- **Desarrollar** confianza en resolución de problemas completos
- **Identificar** áreas de mejora antes de la evaluación

Importante

Estos ejercicios siguen el mismo formato que la prueba, pero con problemas diferentes.

Introducción y Preparación para la Prueba $\ni$ Estructura de la Prueba

Formato de Evaluación

- 3 preguntas de programación aplicada
- Datos proporcionados para cada problema
- Tareas paso a paso claramente especificadas
- Librerías permitidas: `numpy`, `matplotlib`, `scipy`

Temas Evaluados

- Ajuste lineal con `np.polyfit`
- Ajuste no lineal con `scipy.optimize.curve_fit`
- Análisis de algoritmos e instrumentación

Datos Proporcionados para los Ejercicios

Datos Proporcionados para los Ejercicios $\ni$ Datos Proporcionados: Ejercicios 1, 2 y 3



Importante: Ejecutar este código primero

En la prueba, este bloque de código estará disponible al inicio. Ejecútalo antes de resolver cualquier ejercicio.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 # =====
6 # DATOS PARA EJERCICIO 1: Movimiento Rectilíneo Uniforme
7 # =====
8 np.random.seed(42)
9 t_mru = np.linspace(0, 10, 11)
10 x_real = 2.0 + 2.4 * t_mru
11 x_obs = x_real + np.random.normal(0, 0.5, len(t_mru))
12
13 # =====
14 # DATOS PARA EJERCICIO 2: Ley de Gravitación Universal
15 # =====
16 r_grav = np.linspace(1.0, 10.0, 15)
17 K_real = 100.0
18 F_real = K_real / r_grav**2
19 sigma_F = 0.5 + 0.03 * F_real
20 F_obs = F_real + np.random.normal(0, sigma_F)
```

Datos Proporcionados para los Ejercicios $\ni$ Datos Proporcionados: Ejercicio 3 (continuación)



```
1 # =====
2 # DATOS PARA EJERCICIO 3: Análisis de Selection Sort
3 # =====
4 # 8 listas con tamaños 2^2, 2^3, ..., 2^9
5 Lista_1 = np.random.randint(1, 100, size=2**2).tolist()
6 Lista_2 = np.random.randint(1, 100, size=2**3).tolist()
7 Lista_3 = np.random.randint(1, 100, size=2**4).tolist()
8 Lista_4 = np.random.randint(1, 100, size=2**5).tolist()
9 Lista_5 = np.random.randint(1, 100, size=2**6).tolist()
10 Lista_6 = np.random.randint(1, 100, size=2**7).tolist()
11 Lista_7 = np.random.randint(1, 100, size=2**8).tolist()
12 Lista_8 = np.random.randint(1, 100, size=2**9).tolist()
13
14 print("Datos cargados correctamente para los 3 ejercicios")
15 print(f"Ejercicio 1: {len(t_mru)} mediciones de posición vs tiempo")
16 print(f"Ejercicio 2: {len(r_grav)} mediciones de fuerza vs distancia")
17 print(f"Ejercicio 3: 8 listas con tamaños desde {len(Lista_1)} hasta {len(Lista_8)}")
```

Nota: Después de ejecutar este código, todas las variables estarán disponibles para resolver los ejercicios.

Ejercicio 1: Movimiento Rectilíneo Uniforme (np.polyfit)

Ejercicio 1: Movimiento Rectilíneo Uniforme (np.polyfit) $\ni$ Ejercicio 1:



Movimiento Rectilíneo Uniforme

Contexto Físico

Un carrito se mueve sobre un riel de aire con velocidad constante. Se registra su posición en función del tiempo.

Modelo matemático:

$$x(t) = x_0 + v \cdot t$$

donde: x_0 es la posición inicial (m), v es la velocidad (m/s).

Datos Experimentales Proporcionados

- t_{mru} : tiempos de medición (0, 1, 2, ..., 10 segundos)
- x_{obs} : posiciones observadas con ruido experimental



1. Ajuste lineal con np.polyfit:

- Usar `np.polyfit(t_mru, x_obs, deg=1)`
- Extraer velocidad (v) y posición inicial (x_0)
- Calcular posiciones ajustadas: $x_{\text{est}} = x_0 + v \cdot t$

2. Cálculo de R^2 :

- Calcular $SS_{\text{res}} = \sum (x_{\text{obs}} - x_{\text{est}})^2$
- Calcular $SS_{\text{tot}} = \sum (x_{\text{obs}} - \bar{x}_{\text{obs}})^2$
- Obtener $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$

3. Visualización: Gráfico con datos observados (scatter) y ajuste (línea)

4. Reporte: Imprimir x_0 , v y R^2 con formato adecuado

Solución Ejercicio 1

Solución Ejercicio 1 $\Rightarrow$ Solución 1 de Referencia:

Movimiento Rectilíneo Uniforme



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos proporcionados
5 #t_mru
6 #x_obs
7
8 # Tarea 1: Ajuste lineal
9 coef = np.polyfit(t_mru, x_obs, deg=1)
10 v_estimado = coef[0]
11 x0_estimado = coef[1]
12 x_estimado = x0_estimado + v_estimado * t_mru
13
14 # Tarea 2: Cálculo de R²
15 SS_res = np.sum((x_obs - x_estimado)**2)
16 SS_tot = np.sum((x_obs - np.mean(x_obs))**2)
17 R2 = 1 - (SS_res / SS_tot)
```

```
22 # Tarea 3: Visualización
23 plt.figure(figsize=(8, 5))
24 plt.scatter(t_mru, x_obs,
25             label='Datos observados',
26             color='blue', s=50)
27 plt.plot(t_mru, x_estimado, 'r-',
28          linewidth=2, label='Ajuste lineal')
29 plt.xlabel('Tiempo (s)', fontsize=12)
30 plt.ylabel('Posición (m)', fontsize=12)
31 plt.title('Movimiento Rectilíneo Uniforme',
32           fontsize=14)
33 plt.legend()
34 plt.grid(True, alpha=0.3)
35 plt.show()
36
37 # Tarea 4: Reporte
38 print(f"x₀ = {x0_estimado:.3f} m")
39 print(f"v = {v_estimado:.3f} m/s")
40 print(f"R² = {R2:.4f}")
41 calidad = "Bueno" if R2 > 0.95 else "Regular"
42 print(f"Calidad del ajuste: {calidad}")
```

Ejercicio 2: Ley de Gravitación Universal (curve_fit)



Ley de Gravitación Universal

Contexto Físico

Se mide la fuerza gravitacional entre dos masas a diferentes distancias para verificar la ley de gravitación universal.

Modelo matemático:

$$F(r) = \frac{GMm}{r^2}$$

donde: G es la constante gravitacional, M y m son las masas, r es la distancia.

Para simplificar: $F(r) = \frac{K}{r^2}$ donde $K = GMm$

Datos Experimentales Proporcionados

- **r_grav**: distancias (1 a 10 metros)
- **F_obs**: fuerzas observadas con incertidumbres
- **sigma_F**: incertidumbres en las mediciones

Ejercicio 2: Ley de Gravitación Universal (curve_fit) $\ni$ Ejercicio 2: Tareas a Realizar



1. **Definir modelo:** `def modelo_gravitacion(r, K)`
2. **Ajuste no lineal:**
 - Valor inicial: `p0 = [80]`
 - Usar `curve_fit` con ponderación por `sigma_F`
 - Extraer parámetro óptimo y covarianza
3. **Incertidumbres:** Calcular `perr = np.sqrt(np.diag(pcov))`
4. **Cálculo de χ^2 reducido:**
 - Residuos ponderados: $r_i = \frac{F_{\text{obs},i} - F_{\text{fit},i}}{\sigma_{F,i}}$
 - $\chi^2 = \sum r_i^2$, $\nu = n - 1$, $\chi_{\text{red}}^2 = \chi^2 / \nu$
5. **Visualización:** Gráfico con barras de error y curva ajustada
6. **Reporte:** Imprimir $K \pm \sigma_K$ y calidad del ajuste

Solución Ejercicio 2

Solución Ejercicio 2 $\Rightarrow$ Solución 2 de Referencia:

Ley de Gravitación Universal



```
1 from scipy.optimize import curve_fit
2
3 # Datos proporcionados
4 #r_grav
5 #F_obs
6 #sigma_F
7
8 # Tarea 1: Definir modelo
9 def modelo_gravitacion(r, K):
10     return K / r**2
11
12 # Tarea 2: Ajuste no lineal
13 p0 = [80]
14 popt, pcov = curve_fit(modelo_gravitacion,
15                         r_grav, F_obs, p0=p0,
16                         sigma=sigma_F,
17                         absolute_sigma=True)
18 F_fit = modelo_gravitacion(r_grav, *popt)
```

```
29 # Tarea 3: Incertidumbres
30 perr = np.sqrt(np.diag(pcov))
31 sigma_K = perr[0]
32
33 # Tarea 4:  $\chi^2$  reducido
34 residuos = (F_obs - F_fit) / sigma_F
35 chi2 = np.sum(residuos**2)
36 nu = len(r_grav) - 1
37 chi2_red = chi2 / nu
38
39 print(f" $\chi^2 = \{{\rm chi2:.4f}\}")$ 
40 print(f" $\nu = \{{\rm nu}\}")$ 
41 print(f" $\chi^2_{\rm red} = \{{\rm chi2\_red:.4f}\}")$ 
42
43 # Tarea 5: Visualización
44 r_fino = np.linspace(1, 10, 200)
45 F_fino = modelo_gravitacion(r_fino, *popt)
46
47 plt.figure(figsize=(8, 5))
48 plt.errorbar(r_grav, F_obs, yerr=sigma_F, fmt='o',
49              $\hookrightarrow$  label='Datos experimentales')
50 plt.plot(r_fino, F_fino, 'r-',
51          $\hookrightarrow$  linewidth=2, label=f'Ajuste:
52          $\hookrightarrow$   $K=\{{\rm popt[0]:.1f}\} \pm \{{\rm sigma\_K:.1f}\}')$ 
53 plt.xlabel('Distancia (m)', fontsize=12)
54 plt.ylabel('Fuerza (N)', fontsize=12)
55 plt.title('Ley de Gravitación Universal',
56          $\hookrightarrow$  fontsize=14)
57 plt.legend()
58 plt.grid(True, alpha=0.3)
59 plt.show()
```

Ejercicio 3: Análisis del Algoritmo Selection Sort

Ejercicio 3: Análisis del Algoritmo Selection Sort $\ni$ ¿Cómo funciona Selection Sort?

Idea Principal

Selection Sort ordena una lista **seleccionando** repetidamente el elemento más pequeño de la parte no ordenada y colocándolo al inicio.

Proceso Paso a Paso

1. Buscar el elemento **mínimo** en toda la lista
2. Intercambiarlo con el elemento en la **primera posición**
3. Buscar el mínimo en el resto de la lista (desde posición 2)
4. Intercambiarlo con el elemento en la **segunda posición**
5. Repetir hasta que toda la lista esté ordenada

Ejercicio 3: Análisis del Algoritmo Selection Sort $\ni$ Visualización del Proceso de Selection Sort

Ejemplo Visual

Lista inicial: [64, 25, 12, 22, 11]

- Iteración 1: Encontrar $\text{mín}(64,25,12,22,11)=11 \rightarrow [11, 25, 12, 22, 64]$
- Iteración 2: Encontrar $\text{mín}(25,12,22,64)=12 \rightarrow [11, 12, 25, 22, 64]$
- Iteración 3: Encontrar $\text{mín}(25,22,64)=22 \rightarrow [11, 12, 22, 25, 64]$
- Iteración 4: Encontrar $\text{mín}(25,64)=25 \rightarrow [11, 12, 22, 25, 64]$

Ejercicio 3: Análisis del Algoritmo Selection Sort $\ni$ Ejercicio 3:

Análisis del Algoritmo Selection Sort

Contexto Computacional

Selection Sort es otro algoritmo de ordenamiento simple con complejidad $O(n^2)$. Encuentra el elemento mínimo y lo intercambia con la posición actual.

Algoritmo proporcionado:

```
1 def selection_sort(lst):
2     n = len(lst)
3     for i in range(n-1):
4         min_idx = i
5         for j in range(i+1, n):
6             if lst[j] < lst[min_idx]:
7                 min_idx = j
8         lst[i], lst[min_idx] = lst[min_idx], lst[i]
9     return lst
```

Ejercicio 3: Análisis del Algoritmo Selection Sort $\ni$ Ejercicio 3: Datos Proporcionados



Listas Proporcionadas

Se han generado 8 listas con tamaños potencias de 2:

- `Lista_1` con $2^2 = 4$ elementos
- `Lista_2` con $2^3 = 8$ elementos
- `Lista_3` con $2^4 = 16$ elementos
- $\vdots$
- `Lista_8` con $2^9 = 512$ elementos

Ejercicio 3: Análisis del Algoritmo Selection Sort $\ni$ Ejercicio 3:

Tareas a Realizar



1. Instrumentación del algoritmo:

- Crear `selection_sort_instrumentado(lst)`
- Agregar contadores: `comparaciones` e `intercambios`
- Retornar: `(lista_ordenada, comparaciones, intercambios)`

2. Análisis experimental:

- Crear listas: `longitud_lista`, `num_comparaciones`, `num_intercambios`
- Usar bucle para procesar `Lista_1` hasta `Lista_8`
- Almacenar resultados con `.append()`

3. Visualización:

- Gráfico de comparaciones vs tamaño
- Gráfico de intercambios vs tamaño
- Incluir etiquetas, leyenda y título

Solución Ejercicio 3

Solución Ejercicio 3 $\Rightarrow$ Solución 3 de Referencia:

Algoritmo Selection Sort



```
1 # Tarea 1: Instrumentación
2 def selection_sort_instrumentado(lst):
3     n = len(lst)
4     comparaciones = 0
5     intercambios = 0
6
7     for i in range(n-1):
8         min_idx = i
9         for j in range(i+1, n):
10             comparaciones += 1
11             if lst[j] < lst[min_idx]:
12                 min_idx = j
13
14         if min_idx != i:
15             lst[i], lst[min_idx] = \
16                 lst[min_idx], lst[i]
17             intercambios += 1
18
19     return lst, comparaciones, intercambios
20
21 # Tarea 2: Análisis experimental
22 longitud_lista = []
23 num_comparaciones = []
24 num_intercambios = []
```

```
30 for i in range(1, 9):
31     lista = eval(f"Lista_{i}")
32     lista_ord, comp, inter =
        ↪ selection_sort_instrumentado(lista)
33
34     longitud_lista.append(len(lista_ord))
35     num_comparaciones.append(comp)
36     num_intercambios.append(inter)
37
38 # Tarea 3: Visualización
39 plt.figure(figsize=(10, 5))
40 plt.subplot(1, 2, 1)
41 plt.plot(longitud_lista, num_comparaciones, 'o-',
        ↪ color='blue', linewidth=2)
42 plt.xlabel('Tamaño de lista (n)', fontsize=11)
43 plt.ylabel('Comparaciones', fontsize=11)
44 plt.title('Comparaciones vs Tamaño', fontsize=12)
45 plt.grid(True, alpha=0.3)
46
47 plt.subplot(1, 2, 2)
48 plt.plot(longitud_lista, num_intercambios, 's--',
        ↪ color='green', linewidth=2)
49 plt.xlabel('Tamaño de lista (n)', fontsize=11)
50 plt.ylabel('Intercambios', fontsize=11)
51 plt.title('Intercambios vs Tamaño', fontsize=12)
52 plt.grid(True, alpha=0.3)
53
54 plt.tight_layout()
55 plt.show()
56
57 print("Análisis completado")
```

Consolidación y Consejos para la Prueba

Consolidación y Consejos para la Prueba $\ni$ Errores Comunes a Evitar

En Ajustes Lineales (`np.polyfit`)

- Confundir el orden de los coeficientes (pendiente vs intercepto)
- Olvidar calcular las predicciones ajustadas
- No usar `np.mean( )` correctamente para R^2

En Ajustes No Lineales (`curve_fit`)

- No definir correctamente la función del modelo
- Olvidar desempaquetar parámetros con `*popt`
- Confundir `pcov` con las incertidumbres directas
- No calcular grados de libertad correctamente

En Análisis de Algoritmos

- Olvidar incrementar contadores en todos los lugares necesarios
- No usar `.append( )` para almacenar resultados

Consolidación y Consejos para la Prueba $\ni$ Consejos para el Día de la Prueba

Preparación Técnica

- Verificar que todas las librerías funcionen: `numpy`, `matplotlib`, `scipy`
- Tener notebook o script de Python listo para trabajar
- Recordar sintaxis de funciones clave: `polyfit`, `curve_fit`

Estrategia de Resolución

- **Leer completamente** cada ejercicio antes de empezar
- **Seguir el orden** de las tareas especificadas
- **Probar el código** paso a paso (no esperar al final)
- **Comentar** el código para mantener claridad
- **Verificar unidades** en resultados físicos

Gestión del Tiempo

- Distribuir tiempo equitativamente entre las 3 preguntas

Consolidación y Consejos para la Prueba $\ni$ Lista de Verificación Pre-Prueba

Conocimientos Técnicos

- ✓ Ajuste lineal con `np.polyfit`
- ✓ Cálculo de R^2 manualmente
- ✓ Definición de modelos matemáticos en Python
- ✓ Uso de `curve_fit` con incertidumbres
- ✓ Cálculo de χ^2 reducido
- ✓ Instrumentación de algoritmos con contadores
- ✓ Uso de bucles para procesamiento múltiple
- ✓ Creación de gráficos con `matplotlib`

Habilidades Prácticas

- ✓ Lectura e interpretación de enunciados físicos
- ✓ Traducción de fórmulas matemáticas a código
- ✓ Debugging sistemático de errores
- ✓ Interpretación de resultados numéricos

Conclusiones

- **Practicamos** con tres ejercicios tipo prueba:
 - Movimiento rectilíneo uniforme (ajuste lineal)
 - Ley de gravitación universal (ajuste no lineal)
 - Análisis de Selection Sort (instrumentación)
- **Consolidamos** el uso de herramientas clave de Python científico
- **Identificamos** errores comunes y estrategias de resolución
- **Preparamos** el ambiente técnico para la evaluación

Habilidad Desarrollada

Capacidad para resolver problemas completos de física computacional siguiendo especificaciones detalladas, aplicando ajustes de datos y análisis algorítmico.

Antes de la Prueba

- Revisar estos ejercicios de práctica
- Repasar sintaxis de funciones clave
- Verificar instalación de librerías
- Preparar ambiente de trabajo limpio

Durante la Prueba

- Mantener la calma y leer cuidadosamente
- Seguir las tareas paso a paso
- Probar código frecuentemente
- Gestionar bien el tiempo

¡Confía en tu preparación!

Has desarrollado todas las habilidades necesarias